

Project Report: Markdown to Portfolio

Project Title

Portify - A Markdown-to-Portfolio Generator Engine

Members

Batool Zafar (2025181)

Muhammad Hashim (2025530)

Course: CS-112 (OOP in C++)

Instructor: Sir Sajid Ali

Semester: Spring 26

Year: 2026

1. Introduction

“Portify” is an automated pipeline that transforms structured Markdown text into a professional, responsive personal portfolio website. Unlike standard web-based editors, this project utilizes a custom **C++ Object-Oriented Backend** for data parsing and a **Python Flask Bridge** to handle the communication between the web-based frontend and the local file system.

2. Motivation

As Computer Science students, we frequently need to update our portfolios with new projects. Manually writing HTML/CSS for every update is time-consuming and prone to styling errors. We wanted to create a tool where a developer only needs to update a simple text file, and the entire website structure, including navigation, project grids, and contact sections, updates automatically.

3. Purpose

The purpose of **Portify** is to create a seamless bridge between high-level web interfaces and low-level system programming. It provides a specialized tool for developers to generate professional, static portfolios using C++ logic while maintaining the ease of a modern, browser-based Markdown editor.

4. Objectives

- **System Integration:** Establish a reliable data pipeline between JavaScript, Python, and C++.
- **OOP Implementation:** Utilize inheritance and encapsulation to transform raw text into structured HTML components.
- **Sandbox Navigation:** Use a local Python server to bypass browser security restrictions for file I/O tasks.
- **User Experience:** Deliver an IDE-style environment with an adjustable split-screen and real-time previewing.
- **Error Validation:** Implement a robust "fail-fast" system that detects disturbed Markdown structures and alerts the user immediately.

5. Problem Statement

Web browsers operate in a "security sandbox," meaning they cannot natively execute C++ programs or write files to a user's hard drive. Most Markdown editors solve this using purely JavaScript, which bypasses the power of system-level programming. Our challenge was to

build a system that allows a **modern web UI** to trigger a **low-level C++ engine** to process data and generate a physical website locally.

6. Proposed Solution

We developed a **Tri-Layer Architecture**:

1. **Frontend**: A high-performance split-screen editor (HTML/CSS/JS) with an adjustable partition and real-time refresh.
2. **The Bridge (Py-S)**: A local Python server that acts as a middleman, bypassing the browser sandbox to save files and execute the C++ backend.
3. **The Engine (C++ OOP)**: A robust parsing backend that converts raw text into Object-Oriented data structures and injects them into HTML templates.

7. Types of Users

- **Computer Science Students**: To quickly generate portfolios for internships or research project showcases.
- **Minimalists**: Users who want a clean, no-distraction writing environment for their professional bio.

8. Core Programming Concepts Used

This project is a direct application of the CS-112 syllabus. We utilized:

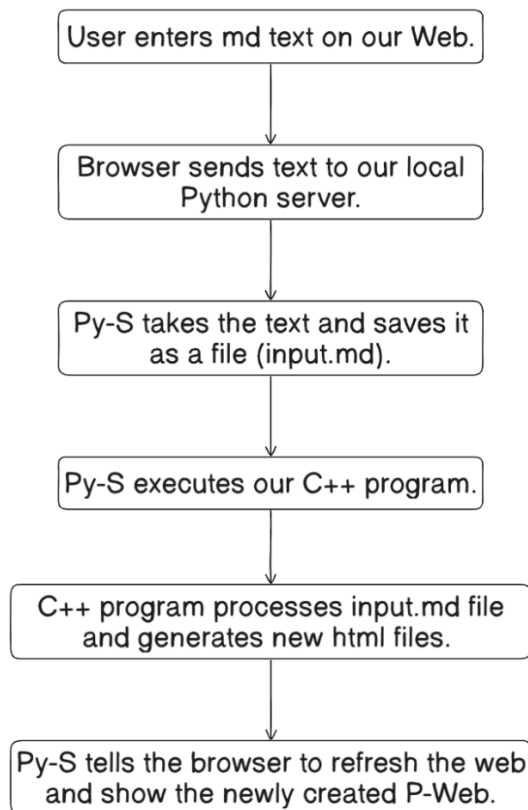
- **Encapsulation**: Protecting data members within classes like Profile, Skills, and Projects.
- **Inheritance**: Using a base class Portfolio to provide common file-handling and string-reading capabilities to all specialized parsing classes.
- **Polymorphism**: A single pointer of 'Portfolio' type pointing to an array containing the dynamically created objects of the sub-classes, hence enforcing late binding or runtime binding.
- **File I/O**: Extensive use of <fstream> to bridge the gap between memory and disk storage.
- **Exception Handling**: Implementing a centralized error-trapping system where the C++ engine communicates "disturbed structure" errors back to the UI via system exit codes.
- **String Manipulation**: Custom parsing logic using std::string functions to locate, extract, and clean data within specific markdown markers.

9. List of Features

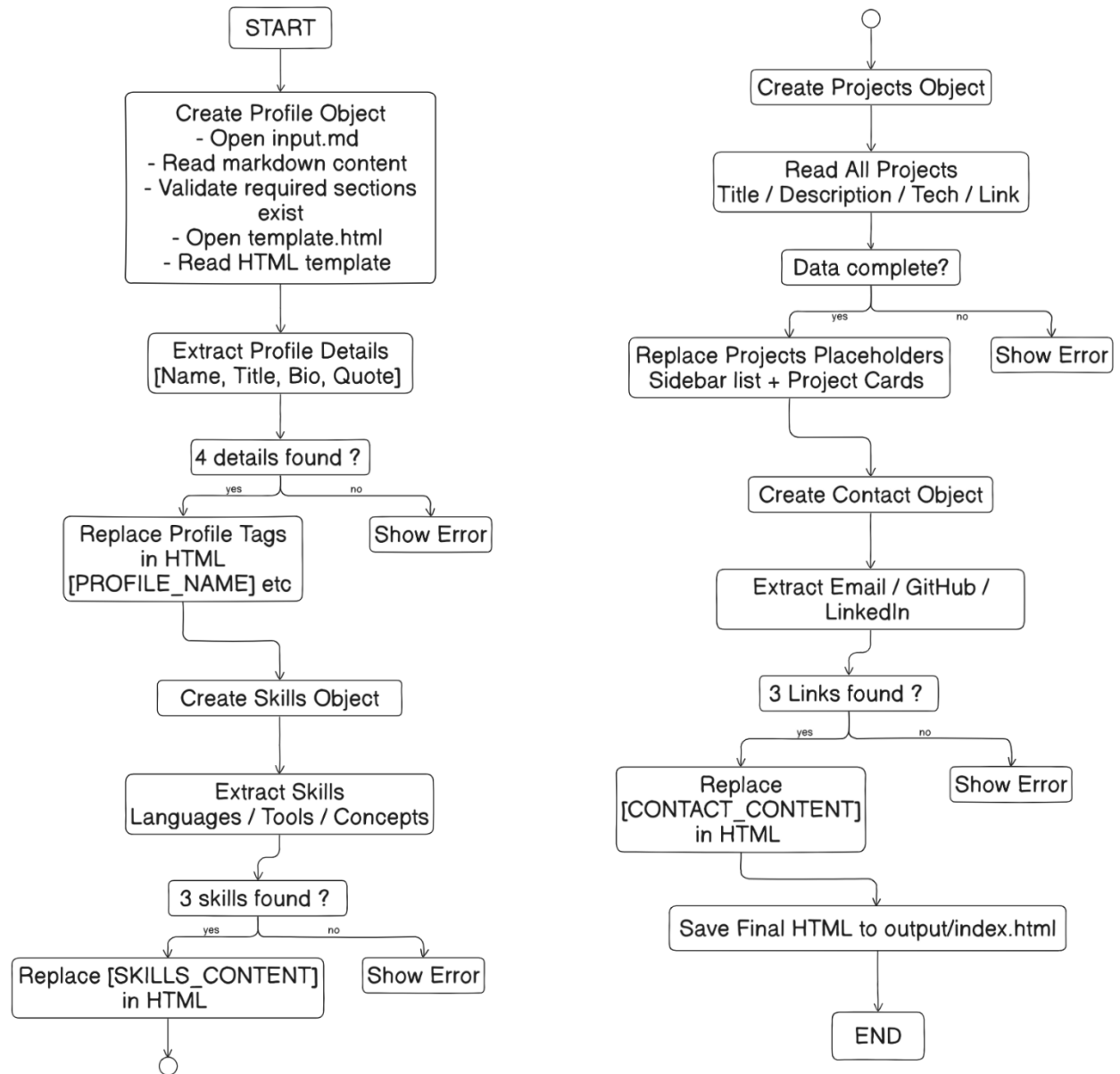
- **Dual-Pane Workspace:** A dark-mode IDE-style editor on the left and a live-viewing iframe on the right.
- **Adjustable Interface:** A draggable vertical partition allowing users to customize their workspace ratio.
- **State-Aware UI:** A "Run" button for local previewing and a "Generate" button that pops the result into a full-screen browser tab.
- **Structured Template Engine:** Logic that handles diverse content types, including profile bios, skill lists, and dynamic project cards.
- **Error Toasting:** An integrated notification system that alerts the user if the Markdown syntax is broken or if the C++ engine encounters a structural error.

10. Flow Diagram & Architecture

Flow Diagram of the engine:



Flow Diagram of C++ Program



11. Contributors

Batool Zafar

- **Frontend Development:** Designed and implemented the user interface for **Portify** (Completely built by AI).

- **Backend Logic:** Developed the complete **Object-Oriented Programming (OOP)** structure.
- **Project Management:** Managed all documentation, header file integration, and authored the project readme.md.

Muhammad Hashim

- **Web Development:** Developed the frontend for **P-web** (Completely built by AI).
- **System Integration:** Built and configured Server.py. (Completely by AI)
- **Backend Logic:** Handled the dynamic polymorphism and completed the 'Projects' class.
- **Documentation:** Authored the final **Project Report**.

12. Conclusion

Portify successfully demonstrates a powerful integration of low-level C++ logic and modern web technologies. By utilizing OOP principles and a Python-based bridge, the project overcomes browser security limitations to offer an efficient, automated portfolio generation tool. It provides a seamless, professional solution for developers to maintain an impactful online presence.

Screenshots

The screenshot displays the Portify Markdown Portfolio Editor interface. The editor is split into two main sections: 'EDITOR' on the left and 'PREVIEW' on the right. The 'EDITOR' section shows a template for a portfolio, with line numbers 1 through 30. The template includes instructions for editing, a profile section with fields for Name, Title, Bio, and a quote, a skills section with fields for Languages, Tools, and Concepts, and a projects section with a field for Project Title 1. The 'PREVIEW' section is currently empty. At the top right of the editor, there is a 'READY' status indicator. At the bottom right, there are two buttons: 'RUN' and 'GENERATE'.

```

1 <!--
2 Instructions:
3 - Edit the text in just the square brackets.
4 - For links, paste the url in the round brackets.
5 - To add more projects, just copy paste the 5 lines.
6 - If you can't see your desired output, try reloading the website or
7 -->
8
9
10 # PROFILE
11
12 Name: [Your Full Name]<br>
13 Title: [Your Professional Title, e.g., Software Engineer]<br>
14 Bio: [Write a short introduction about yourself, your background, and yo
15
16 > "[Insert a personal quote, philosophy, or objective statement.]"
17
18
19 # SKILLS
20 * Languages: [List your programming languages, e.g., C++, Java, Python]
21 * Tools: [List your tools, e.g., Git, VS Code, Linux]
22 * Concepts: [List key concepts, e.g., OOP, Data Structures, Web Develop
23
24 # PROJECTS
25
26 ## [Project Title 1]
27 Title: [Title of your project.]<br>
28 Description: [Brief description of what the project does and your role i

```

